

DATA203 Foundational Python (Prof. Maull) / Fall 2026 / HW0

Points Possible	Due Date	Time Commitment (estimated)
10	Sunda, September 6 @ midnight	up to 8 hours

- **GRADING:** Grading will be aligned with the completeness of the objectives.
- **INDEPENDENT WORK:** Copying, cheating, plagiarism and academic dishonesty *are not tolerated* by University or course policy. Please see the syllabus for the full departmental and University statement on the academic code of honor.

OBJECTIVES

- Familiarize yourself with the JupyterLab environment, Markdown and Python
- Familiarize yourself with Github and basic git
- Explore JupyterHub Linux terminal console integrating what you learned in the prior parts of this homework
- Learn more about and use string manipulation

WHAT TO TURN IN

You are being encouraged to turn the assignment in using the provided Jupyter Notebook. To do so, make a directory in your Lab environment called `homework/hw0`. Put all of your files in that directory. Then zip or tar that directory, rename it with your name as the first part of the filename (e.g. `maull_hw0_files.zip`, `maull_hw0_files.tar.gz`), then download it to your local machine, then upload the `.zip` to Canvas.

If you do not know how to do this, please ask, or visit one of the many tutorials out there on the basics of using zip in Linux.

If you choose not to use the provided notebook, you will still need to turn in a `.ipynb` Jupyter Notebook and corresponding files according to the instructions in this homework.

ASSIGNMENT TASKS

(0%) Familiarize yourself with the JupyterLab environment, Markdown and Python

As stated in the course announcement [Jupyter \(https://jupyter.org\)](https://jupyter.org) is the core platform we will be using in this course and is a popular platform for data scientists around the world. We have a JupyterLab setup for this course so that we can operate in a cloud-hosted environment, free from some of the resource constraints of running Jupyter on your local machine (though you are free to set it up on your own and seek my advice if you desire).

You have been given the information about the Jupyter environment we have setup for our course, and the underlying Python environment will be using is the [Anaconda \(https://anaconda.com\)](https://anaconda.com) distribution. It is not necessary for this assignment, but you are free to look at the multitude of packages installed with Anaconda, though we will not use the majority of them explicitly.

As you will soon find out, Notebooks are an incredibly effective way to mix code with narrative and you can create cells that are entirely code or entirely Markdown. Markdown (MD or `md`) is a highly readable text format that allows for easy documentation of text files, while allowing for HTML-based rendering of the text in a way that is style-independent.

We will be using Markdown frequently in this course, and you will learn that there are many different “flavors” of Markdown. We will only be using the basic flavor, but you will benefit from exploring the “Github flavored” Markdown, though you will not be responsible for using it in this course – only the “basic” flavor. Please refer to the original course announcement about Markdown.

\$ Task: THERE IS NOTHING TO TURN IN FOR THIS PART.

Play with and become familiar with the basic functions of the Lab environment given to you online in the course.

\$ Task: THERE IS NOTHING TO TURN IN FOR THIS PART.

Please *create a markdown document* and read the documentation for basic Markdown [here](#). Learn to use all of the following:

- headings (one level is fine),
- bullets,
- bold and italics

Again, the content of your document can be whatever you like, just learn some of the basic functionality of Markdown.

(0%) Familiarize yourself with Github and basic git

Github (<https://github.com>) is the *de facto* platform for open source software in the world based on the very popular [git](https://git-scm.org) (<https://git-scm.org>) version control system. Git has a sophisticated set of tools for version control based on the concept of local repositories for fast commits and remote repositories only when collaboration and remote synchronization is necessary. Github enhances git by providing tools and online hosting of public and private repositories to encourage and promote sharing and collaboration. Github hosts some of the world's most widely used open source software.

If you are already familiar with git and Github, then this part will be very easy!

\$ Task: Create a public Github repo named "hu-f26-data203" and place a README.md file in it.

Create your first file called README.md at the top level of the repository.

Please put your Github username in the file. Aside from that you can put whatever text you like in the file (If you like, use something like [lorem ipsum](#) to generate random sentences to place in the file.). Please include the link to **your** Github repository that now includes the minimal README.md. You don't have to have anything elaborate in that file or the repo.

\$ Task: Fork the course repository.

Learn to use Github workflows and fork the class repo:

- https://github.com/kmhuads/f26_data203

(0%) Explore JupyterHub Linux terminal console integrating what you learned in the prior parts of this homework

The Linux console in JupyterLab is a great way to perform command-line tasks and is an essential tool for basic scripting that is part of a data scientist's toolkit. Open a terminal console in the lab environment and familiarize yourself with your files and basic commands.

\$ Task: Understand basic Linux file operations

Basic file operations go a long way to understand the way Linux works. In this part, you will understand folders, files and making revisions to a file. These files will be visible within Jupyter, which makes moving from one platform to another seamless. We will create a folder, file, make edits.

- open a Jupyter console
- create a file called README.md
- type `mkdir your_folder_name` to create a folder in filesystem *in the current folder where you are*
- use `cd your_folder_name` to "change directory" and move into the folder you just created
- use `pwd` to "print working directory" to verify you are in the folder you created
- create a file by type `touch README.md` the touch command creates a file if it does not already exist, otherwise it will change the timestamp of that file when it is "touched"
- type `echo "Hello this is test text." > README.md`. This will take the words you typed and "append them" into the file README.md
- to see the contents of your file typing `cat README.md` or `more README.md` or `less README.md`

\$ Task: Learn to quickly obtain remote files in Linux

The commands `wget` and `curl` are useful for grabbing data and files from remote resources off the web. Using these tools from the command line streamline your workflows and are often faster than writing a program to do the same. These tools will also expand and strengthen your data science skills, adding a few more tools to your toolkit is rarely a bad idea.

1. Read the documentation on each of these commands by typing `man wget` or `man curl` in the terminal.

- `man` stands for *manual* and nearly all versions of Linux have such documentation pages for the majority of commands. If it fails, try the command with the `-h` or `--help` flag, such as `wget --help`
- 2. Make sure your output goes to a file and study the documentation to select the proper flags to do so.
- 3. You can obtain nearly any file anywhere on the Internet with these commands. For example, the Library of Congress “Off the Record” interview with the late, great Quincy Jones from 1988: <https://www.loc.gov/item/jsmith000198/>.

Follow the steps below:

- click on this interview link
- choose the dropdown for the **mp3**
- when the page opens up, there will be a player that starts the interview, copy the URL
- go to your Jupyter terminal and run the command `wget <the_url_just_copied>`, where you will paste the URL you just copied in the `<the_url_just_copied>`
- 4. Either on the Library of Congress site or somewhere else, play further with `wget` and `curl` to download some other files you might have of interest.

(100%) Learn more about and use string manipulation

We learned in lecture that strings are *sequences* in Python.

We also learned that the sequence types have a number of basic operations like concatenation, length, etc.

Strings have a lot of other operations on them that make them exceedingly useful for text processing. In fact, Python is exceptionally good at processing text, as you will see.

You will need to use the provided **starter notebook** here:

- [hw0_starter.ipynb](#)

First things first, please go to the documentation on Python strings, also known as `str` or “Text Sequence Type”:

- <https://docs.python.org/3/library/stdtypes.html#textseq>

Study it and especially the String methods (see more here: <https://docs.python.org/3/library/stdtypes.html#string-methods>).

You will be using this short **AI-generated writeup (gpt-oss/120B; note → there may be inaccuracies in the generated text)** about the unimitable jazz saxophonist Hank Mobley.

Henry “Hank” Mobley was born on April 28 1930 in Greenville, Georgia, into a family where music was a daily presence. He began playing the saxophone as a child, first on the alto before switching to tenor in his teens. After moving to New York City in the late 1940s, Mobley immersed himself in the burgeoning bebop scene, absorbing lessons from Charlie Parker and securing early professional gigs with the Jazz At-The-Philharmonic tours. By the mid-1950s his warm, resonant tone and lyrical phrasing caught the attention of several prominent producers, leading to his first recording contracts and a rapid rise within the hard-bop community.

During the late 1950s and early 1960s Mobley became one of the most prolific tenor saxophonists on the recording circuit, releasing a string of classic albums that are still regarded as essential listening. Notable sessions include *The Turnaround!* (1955), *Soul Station* (1960), *Roll Call* (1960) and *Workout* (1961). These dates showcase his dual talent as a composer and improviser; original tunes such as “The Breeze,” “Roll Call,” and “The Feelin’” combine memorable melodic lines with sophisticated, blues-inflected harmonies. In addition to leading his own dates, Mobley was a highly sought-after sideman, contributing expressive solos to landmark recordings by Art Blakey’s Jazz Messengers, Horace Silver, Lee Morgan, Grant Green, and many others.

Mobley’s “relaxed, swinging tone” and “economical yet expressive phrasing” helped shape the hard-bop aesthetic of the era. While his playing was often described as understated compared with the more fiery styles of contemporaries like Dexter Gordon or John Coltrane, critics praised his ability to convey deep feeling without excess. His melodic sensibility and compositional craft influenced a generation of tenor players—including Wayne Shorter, Joe Dixon, and later modern voices such as Joshua Redman—who view Mobley as a template for blending groove, lyricism, and harmonic sophistication.

Personal struggles, including health issues and financial instability, curtailed Mobley's recording activity after the mid-1960s, and he gradually withdrew from the public eye. He passed away on May 30 1986, but his discography has been continuously reissued and remastered, often accompanied by new liner notes that contextualize his contributions to jazz history. Today, Mobley's recordings remain a cornerstone of the hard-bop canon, and his reputation has grown steadily among scholars, musicians, and listeners who regard him as one of the most melodic and technically refined tenor saxophonists of his generation.

§ Task: Basic String functions.

Use the passage provided and answer the questions. **Provide your answer in THIS Jupyter Notebook.**

1. Assign a variable called `passage` with the string provided into a multi-line string using `"""` which we talked about in lecture.
Make sure you preserve the spaces between paragraphs, which are `"\n\n"` (two newlines).
2. Use `lower()` to produce the lowercase version of the whole string.
3. Use `str.split()` to determine how many words are in the string.
Recall, `split()` returns a *sequence* (i.e. a list sequence), so counting the words is easier with `len()` on that returned sequence.
4. **BONUS** (up to 2 bonus points): write the code to count the number of unique words. Your solution must eliminate punctuation to be considered for full points.

§ Task: Advanced String functions.

Now that you have `passage` in a variable, there are a few more String operations we want to try to familiarize ourselves with.

1. Use `replace()` to replace all instances of the words "he" and "his" with `###`.
You would be advised to use `str.lower()` as in the prior code first so that all of your words are *normalized*. **Note also that you will need to ADD the spaces around the word (e.g. " he " or `replace()` will turn words like there into t#re, which is not what you want.** So for example, the sentence "He passed away on May 30 1986, but his discography has been continuously reissued and remastered, ..." would be "### passed away on May 30 1986, but ### discography has been continuously reissued and remastered, ...".
2. Write the Python code to count how many "and" words are in the passage? Study the `str.count()` function.
3. How many times was the word "jazz" used in the passage? Use the lowercase-normalized count.

§ Task: Sequence introduction with built-in functions.

Now we will put looping into our work and ask more complex questions of the text. Some code is provided to help you with this task.

Next will will a list of all the words in `passage` then you will learn that `str.split()` will be very valuable in doing that.

1. Study what `passage.split()` does and **write a sentence fragment explaining what it does.**
2. Get the first paragraph (`passage` index 0), and **return the number of words in the paragraph.**
 - **NOTE:** you can include punctuation as part of the word.
 - You will make use of `split()` TWICE once to split on two newlines or `\n\n` for the paragraph and once for the paragraph's words.
 - You will then utilize the built-in function `len()`. DO NOT overthink this.

Now use this to complete the task.

§ Task: Sequence iteration with for loops.

Now we will put looping into our work and ask more complex questions of the text. This will be a stretch since we have not quite yet completed the full knowledge of looping, but this is a nice intro.

You previously noted that a list of all the words in `passage` then can be produced with `str.split()` on the first paragraph once it is obtained.

There is template code to get you started. You will use it and complete it to produce a count of all the words of length 5 in the FIRST paragraph.

1. Get the first paragraph (index 0) as before. **NOTE:** you can include punctuation as part of the word. You will make use of your solution in #2 and `split()`.
2. How many words of length 5 are in the first paragraph.